

U2-AT2 PYTHON CODE

1. PYTHON CODE:

```
from heapq import heappush, heappop

# Graph stores drone paths and their travel costs
graph = {}

# Heuristic values (estimated distance to goal)
heuristic = {}

# Input number of locations
n = int(input("Enter number of locations: "))

# Input locations and heuristic values
for i in range(n):
    node = input("Enter location name: ")
    h = int(input("Enter heuristic value: "))

    heuristic[node] = h
    graph[node] = []

# Input routes
edges = int(input("Enter number of routes: "))
```

```
for i in range(edges):
```

```
    start, end, cost = input(  
        "Enter route (from to cost): "  
    ).split()
```

```
    graph[start].append((end, int(cost)))
```

```
# Starting point and destination
```

```
source = input("Enter starting location: ")
```

```
goal = input("Enter destination location: ")
```

```
# A* Search Algorithm
```

```
def a_star(start, goal):
```

```
    # Priority queue contains:
```

```
    # (f(n), g(n), current node, path)
```

```
    priority_queue = []
```

```
    heappush(  
        priority_queue,  
        (heuristic[start], 0, start, [start])  
    )
```

```
visited = set()
```

```
while priority_queue:
```

```
    f, g, current, path = heappop(priority_queue)
```

```
    # Goal reached
```

```
    if current == goal:
```

```
        return path, g
```

```
    if current not in visited:
```

```
        visited.add(current)
```

```
        # Explore neighboring locations
```

```
        for neighbour, cost in graph[current]:
```

```
            new_g = g + cost
```

```
            new_f = new_g + heuristic[neighbour]
```

```
            heappush(
```

```
                priority_queue,
```

```

        (
            new_f,
            new_g,
            neighbour,
            path + [neighbour]
        )
    )
)

```

```

return None

```

```

# Calling AI algorithm

```

```

result = a_star(source, goal)

```

```

print("\nOptimal Drone Route:", result[0])

```

```

print("Minimum Travel Cost:", result[1])

```

2. PYTHON CODE:

```

def fitness(x):

```

```

    return -(x-35)**2+100

```

```

start=int(input("Enter initial signal timing: "))

```

```

def hill_climbing(start):

```

```
current=start
```

```
while True:
```

```
    neighbors=[  
        current-1,  
        current+1  
    ]
```

```
    best=current
```

```
    for value in neighbors:
```

```
        if fitness(value)>fitness(best):  
            best=value
```

```
    if best==current:
```

```
        return current,fitness(current)
```

```
    current=best
```

```
result=hill_climbing(start)
```

```
print(result)
```

3. PYTHON CODE:

```
graph={}
```

```
nodes=int(input("Enter number of locations: "))
```

```
for i in range(nodes):
```

```
    node=input("Location name: ")
```

```
    graph[node]=input(
```

```
        "Enter connected locations: "
```

```
    ).split()
```

```
hazards=input(
```

```
"Enter hazard locations: "
```

```
).split()
```

```
start=input("Enter start location: ")
```

```
goal=input("Enter goal location: ")
```

```
def rover(start,goal):
```

```
    visited=[]
```

```
    def search(node):
```

```
visited.append(node)
```

```
if node==goal:
```

```
    return True
```

```
for nxt in graph[node]:
```

```
    if nxt not in visited and nxt not in hazards:
```

```
        if search(nxt):
```

```
            return True
```

```
return False
```

```
search(start)
```

```
return visited
```

```
print(rover(start,goal))
```

4. PYTHON CODE:

```
subjects=input(
```

```
"Enter subjects: "
```

```
).split()
```

```
slots=input(
```

```
"Enter available slots: "
```

```
).split()
```

```
solution={}
```

```
def valid(slot):
```

```
    return slot not in solution.values()
```

```
def solve(index):
```

```
    if index==len(subjects):
```

```
        return True
```

```
    subject=subjects[index]
```

```
    for slot in slots:
```



```
if valid(slot):
```

```
    solution[subject]=slot
```

```
    if solve(index+1):
```

```
        return True
```

```
    del solution[subject]
```

```
return False
```

```
solve(0)
```

```
print(solution)
```

5. PYTHON CODE:

```
values=list(  
map(int,input(  
"Enter game values: "  
).split()  
)
```

```
def minimax(depth,index,maximize,alpha,beta):
```

```
    if depth==2:
```

```
        return values[index]
```

```
    if maximize:
```

```
        best=-999
```

```
        for i in range(2):
```

```
            best=max(
```

```
                best,
```

```
                minimax(
```

```
                    depth+1,
```

```
                    index*2+i,
```

```
                    False,
```

```
                    alpha,
```

```
                    beta
```

```
                )
```

```
            )
```

```
        alpha=max(alpha,best)
```

```
    if alpha>=beta:
```

break

return best

else:

best=999

for i in range(2):

```
best=min(  
    best,  
    minimax(  
        depth+1,  
        index*2+i,  
        True,  
        alpha,  
        beta  
    )  
)
```

beta=min(beta,best)

if alpha>=beta:

```
break
```

```
return best
```

```
result=minimax(  
0,0,True,-999,999  
)
```

```
print(result)
```